

MODULE V

File System

MODULE V

- File System: File concept - Attributes, Operations, types, structure – Access methods, Protection. File-system implementation, Directory implementation. Allocation methods.
- Storage Management: Magnetic disks, Solid-state disks, Disk Structure, Disk scheduling, Disk formatting.

File Concept

- A file is a named collection of related information that is recorded on secondary storage.
- A file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user.
- The information in a file is defined by its creator.
- Many different types of information may be stored in a file—source or executable programs, numeric or text data, photos, music, video, and so on.

File Concept

- A file has a certain defined structure, which depends on its type.
- A text file is a sequence of characters organized into lines (and possibly pages).
- A source file is a sequence of functions, each of which is further organized as declarations followed by executable statements.
- An executable file is a series of code sections that the loader can bring into memory and execute.

File Attributes

- A file's attributes vary from one operating system to another but typically consist of these:
- Name. The symbolic file name is the only information kept in human readable form.
- Identifier. This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.
- Type. This information is needed for systems that support different types of files.
- Location. This information is a pointer to a device and to the location of the file on that device.
- Size. The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- Protection. Access-control information determines who can do reading, writing, executing, and so on.
- Time, date, and user identification. This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

File Operations

- **Creating a file.** Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.
- **Writing a file.** To write a file, we make a system call specifying both
the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the file's location. The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.

File Operations

- **Reading a file.** To read from a file, we use a system call that specifies the name of the file and where (in memory) the next block of the file should be put.
- Again, the directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place.
- Once the read has taken place, the read pointer is updated. Because a process is usually either reading from or writing to a file, the current operation location can be kept as a per-process current file-position pointer.
- Both the read and write operations use this same pointer, saving space and reducing system complexity.

File Operations

- **Repositioning within a file.** The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value.
- Repositioning within a file need not involve any actual I/O. This file operation is also known as a file seek.
- **Deleting a file.** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.

File Operations

- **Truncating a file.** The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged—except for file length—but lets the file be reset to length zero and its file space released.
- Most of the file operations mentioned involve searching the directory for the entry associated with the named file. To avoid this constant searching, many systems require that an `open()` system call be made before a file is first used. The operating system keeps a table, called the open-file table, containing information about all open files. When a file operation is requested, the file is specified via an index into this table, so no searching is required. When the file is no longer being actively used, it is closed by the process, and the operating system removes its entry from the open-file table.

File Operations

- Some systems implicitly open a file when the first reference to it is made. The file is automatically closed when the job or program that opened the file terminates.
- The operating system uses two levels of internal tables: a per-process table and a system-wide table.
- The per-process table tracks all files that a process has open. Stored in this table is information regarding the process's use of the file.. Access rights to the file and accounting information can also be included.

File Operations

- Each entry in the per-process table in turn points to a system-wide open-file table.
- The system-wide table contains process-independent information, such as the location of the file on disk, access dates, and file size.
- Once a file has been opened by one process, the system-wide table includes an entry for the file.
- The open-file table also has an open count associated with each file to indicate how many processes have the file open.

File Operations

Information associated with an open file

- File pointer- The system must track the last read– write location as a current-file-position pointer. This pointer is unique to each process operating on the file and therefore must be kept separate from the on-disk file attributes.
- File-open count- Multiple processes may have opened a file, and the system must wait for the last file to close before removing the open-file table entry. The file-open count tracks the number of opens and closes and reaches zero on the last close. The system can then remove the entry.

File Operations

- Disk location of the file. Most file operations require the system to modify data within the file. The information needed to locate the file on disk is kept in memory so that the system does not have to read it from disk for each operation.
- Access rights. Each process opens a file in an access mode. This information is stored on the per-process table so the operating system can allow or deny subsequent I/O requests.

File Types

- A common technique for implementing file types is to include the type as part of the file name. The name is split into two parts—a name and an extension, usually separated by a period. In this way, the user and the operating system can tell from the name alone what the type of a file is.
- Most operating systems allow users to specify a file name as a sequence of characters followed by a period and terminated by an extension made up of additional characters. Examples include `resume.docx`, `server.c`, and `ReaderThread.cpp`.

File Types

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

Figure 11.3 Common file types.

File Structure

- File types also can be used to indicate the internal structure of the file.
- Some operating systems consists of a set of system-supported file structures, with sets of special operations for manipulating files with those structures.
- If the operating system defines different file structures, it needs to contain the code to support these file structures. In addition, it may be necessary to define every file as one of the file types supported by the operating system.
- When new applications require information structured in ways not supported by the operating system, severe problems may result

File Structure

- Some operating systems impose (and support) a minimal number of file structures. This approach has been adopted in UNIX, Windows, and others.
- Each application program must include its own code to interpret an input file as to the appropriate structure. However, all operating systems must support at least one structure—that of an executable file—so that the system is able to load and run programs.

Access Methods

- The information in the file can be accessed in several ways.
- **Sequential Access**-Information in the file is processed in order, one record after the other.
- example, editors and compilers usually access files in this fashion.
- A read operation—`read next()`—reads the next portion of the file and automatically advances a file pointer, which tracks the I/O location.
- The write operation—`write next()`—appends to the end of the file and advances to the end of the newly written material (the new end of file).

Access Methods

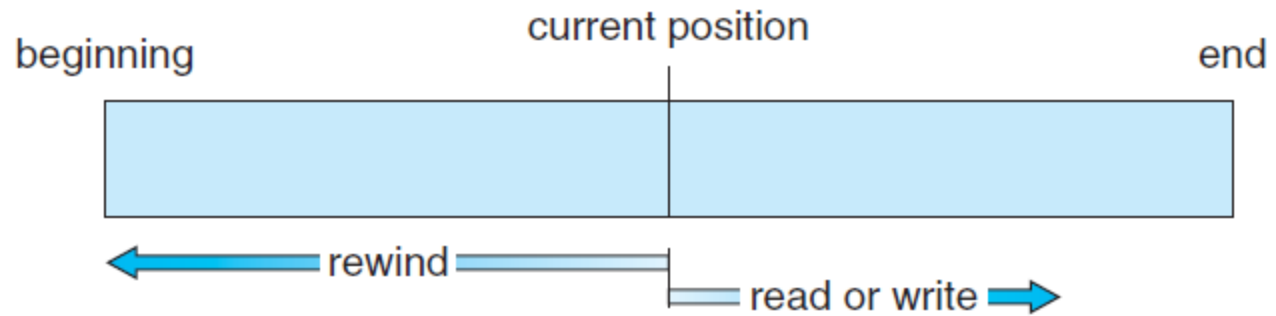


Figure 11.4 Sequential-access file.

Access Methods

- **Direct Access**-A file is made up of fixed-length **logical records that allow programs to read and write records** rapidly in no particular order.
- The file is viewed as a numbered sequence of blocks or records. Thus, we may read block 14, then read block 53, and then write block 7. There are no restrictions on the order of reading or writing for a direct-access file.
- For the direct-access method, the file operations must be modified to include the block number as a parameter.

Access Methods

- ie, `read(n)`, where `n` is the block number, rather than `read next()`, and `write(n)` rather than `write next()`.
- An alternative approach is to retain `read next()` and `write next()`, as with sequential access, and to add an operation position `file(n)` where `n` is the block number. Then, to effect a `read(n)`, we would position `file(n)` and then `read next()`.

Access Methods

sequential access	implementation for direct access
reset	cp = 0;
read_next	read cp ; cp = cp + 1;
write_next	write cp ; cp = cp + 1;

Figure 11.5 Simulation of sequential access on a direct-access file.

Access Methods

- **Index access method-** like an index in the back of a book, contains pointers to the various blocks. To find a record in the file, we first search the index and then use the pointer to access the file directly and to find the desired record.

Protection

- File systems can be damaged by hardware problems (such as errors in reading or writing), power surges or failures, head crashes, dirt, temperature extremes, and vandalism.
- Reliability is generally provided by duplicate copies of files. Many computers have systems programs that automatically (or through computer-operator intervention) copy disk files to tape at regular intervals (once per day or week or month) to maintain a copy should a file system be accidentally destroyed.
- A single-user laptop system, we might provide protection by locking the computer in a desk drawer or file cabinet.

Protection

- **Types of Access**
- Protection mechanisms provide controlled access by limiting the types of file access that can be made.
- Access is permitted or denied depending on several factors, one of which is the type of access requested. Several different types of operations may be controlled :**Read, Write, Execute, Append, Delete, List**
- **Access Control**- Make file access dependent on the identity of the user.
- The most general scheme to implement identity dependent access is to associate with each file and directory an **access-control list (ACL) specifying user names and the types of access allowed for each user.**

Protection

- When a user requests access to a particular file, the operating system checks the access list associated with that file.
- If that user is listed for the requested access, the access is allowed. Otherwise, a protection violation occurs, and the user job is denied access to the file.
- **Disadvantages**
 - Constructing such a list may be a tedious and unrewarding task, especially if we do not know in advance the list of users in the system.
 - The directory entry, previously of fixed size, now must be of variable size, resulting in more complicated space management.

Protection

- To condense the length of the access-control list, many systems recognize three classifications of users in connection with each file:
- Owner. The user who created the file is the owner.
- Group. A set of users who are sharing the file and need similar access is a group, or work group.
- Universe. All other users in the system constitute the universe.
- With the more limited protection classification, only three fields are needed to define protection.
- Each field is a collection of bits, and each bit either allows or prevents the access associated with it.
- For example, the UNIX system defines three fields of 3 bits each—rwx, where r controls read access, w controls write access, and x controls execution.

Protection

- **Other Protection Approaches**

- Another approach to the protection problem is to associate a password with each file.
- Disadvantages
 - The number of passwords that a user needs to remember may become large, making the scheme impractical.
 - If only one password is used for all the files, then once it is discovered, all files are accessible; protection is on an all-or-none basis.

File-system implementation

- A file system is a method of organizing files on physical media, such as hard disks, CD's, and flash drives.
- Several on-disk and in-memory structures are used to implement a file system.
- These structures vary depending on the operating system and the file system.
- **On disk structures**
 - **Boot control block** (per volume) can contain information needed by the system to boot an operating system from that volume. It is typically the first block of a volume. In UFS, it is called the boot block. In NTFS, it is the partition boot sector.

File-system implementation

- A **volume control block** (per volume) contains volume (or partition) details, such as the number of blocks in the partition, the size of the blocks, a free-block count and free-block pointers, and a free-FCB count and FCB pointers. In UFS, this is called a superblock. In NTFS, it is stored in the master file table.
- A **directory structure** (per file system) is used to organize the files. In UFS, this includes file names and associated inode numbers. In NTFS, it is stored in the master file table.
- A **per-file FCB** contains many details about the file. It has a unique identifier number to allow association with a directory entry.

File-system implementation

- **in-memory structures**

- An in-memory **mount table** contains information about each mounted volume.
- An in-memory **directory-structure** cache holds the directory information of recently accessed directories.
- The **system-wide open-file table** contains a copy of the FCB of each open file, as well as other information.
- The **per-process open-file table** contains a pointer to the appropriate entry in the system-wide open-file table, as well as other information.
- **Buffers** hold file-system blocks when they are being read from disk or written to disk.

File-system implementation

- **To create a new file**
 - It allocates a new FCB.
 - The system then reads the appropriate directory into memory, updates it with the new file name and FCB, and writes it back to the disk.
- **To open a file**
 - The open() call passes a file name to the logical file system.
 - The open() system call first searches the system-wide open-file table to see if the file is already in use by another process.
 - If it is, a per-process open-file table entry is created pointing to the existing system-wide open-file table.

File-system implementation

- If the file is not already open, the directory structure is searched for the given file name. Parts of the directory structure are usually cached in memory to speed directory operations.
- Once the file is found, the FCB is copied into a system-wide open-file table in memory.
- Next, an entry is made in the per-process open-file table, with a pointer to the entry in the system-wide open-file table and some other fields. These
- other fields may include a pointer to the current location in the file (for the next read() or write() operation) and the access mode in which the file is open.

File-system implementation

- The `open()` call returns a pointer to the appropriate entry in the per-process file-system table.
- **To close a file**
 - When a process closes the file, the per-process table entry is removed, and the system-wide entry's open count is decremented.
 - When all users that have opened the file close it, any updated metadata is copied back to the disk-based directory structure, and the system-wide open-file table entry is removed.

File-system implementation

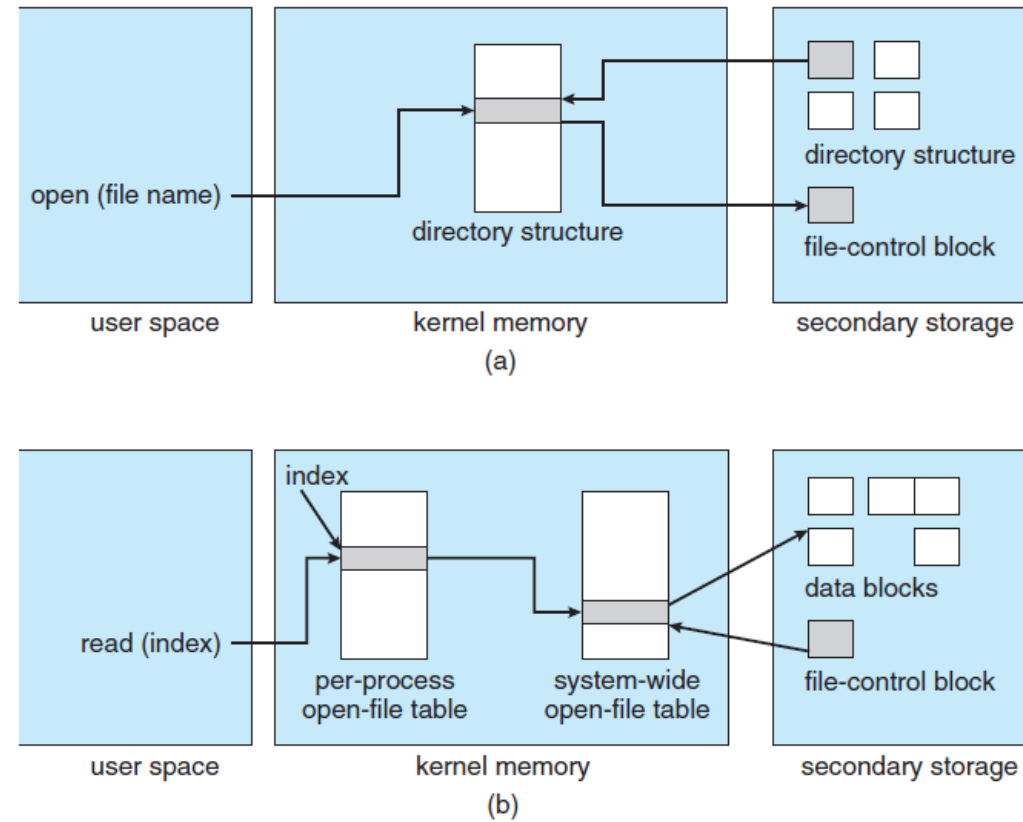


Figure 12.3 In-memory file-system structures. (a) File open. (b) File read.

Directory Implementation

- **Linear List**
- Directory is a linear list of file names with pointers to the data blocks.
- To create a new file, we must first search the directory to be sure that no existing file has the same name. Then, we add a new entry at the end of the directory.
- To delete a file, we search the directory for the named file and then release the space allocated to it.
- To reuse the directory entry, we can do one of several things. We can mark the entry as unused (by assigning it a special name, such as an all-blank name, or by including a used– unused bit in each entry), or we can attach it to a list of free directory entries.

Directory Implementation

- A third alternative is to copy the last entry in the directory into the freed location and to decrease the length of the directory.
- The real disadvantage of a linear list of directory entries is that finding a file requires a linear search.
- Many operating systems implement a software cache to store the most recently used directory information. A cache hit avoids the need to constantly reread the information from disk.
- A sorted list allows a binary search and decreases the average search time.

Directory Implementation

- **Hash Table**

- A linear list stores the directory entries, but a hash data structure is also used. The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list.
- The major difficulties with a hash table are its generally fixed size and the dependence of the hash function on that size.
- We can use a chained-overflow hash table. Each hash entry can be a linked list instead of an individual value, and we can resolve collisions by adding the new entry to the linked list.

Allocation Methods

- Allocation Methods deals with how to allocate space to the files so that disk space is utilized effectively and files can be accessed quickly.
- Three major methods of allocating disk space are in wide use:
 - contiguous
 - linked
 - Indexed

Allocation Methods

Contiguous Allocation

- Contiguous allocation requires that each file occupy a set of contiguous blocks on the disk.
- Disk addresses define a linear ordering on the disk. With this ordering, assuming that only one job is accessing the disk, accessing block $b + 1$ after block b normally requires no head movement.
- When head movement is needed the head need only move from one track to the next.
- Thus, the number of disk seeks required for accessing contiguously allocated files is minimal, as is seek time when a seek is finally needed.

Allocation Methods

- Contiguous allocation of a file is defined by the disk address and length (in block units) of the first block.
- If the file is n blocks long and starts at location b , then it occupies blocks b , $b + 1$, $b + 2$, ..., $b + n - 1$.
- The directory entry for each file indicates the address of the starting block and the length of the area allocated for this file .
- Both sequential and direct access can be supported by contiguous allocation.

Allocation Methods

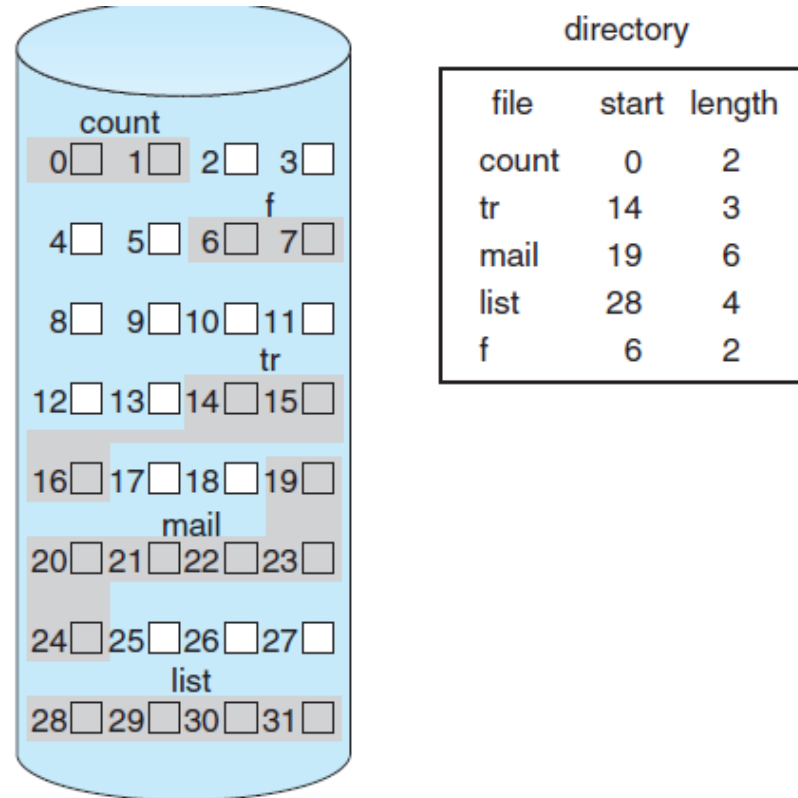


Figure 12.5 Contiguous allocation of disk space.

Allocation Methods

- **Disadvantage**
- **Finding space for a new file.**
- First fit and best fit are the most common strategies used to select a free hole from the set of available holes.
- All these algorithms suffer from the problem of **external fragmentation**.
- As files are allocated and deleted, the free disk space is broken into little pieces.
- One strategy for preventing loss of significant amounts of disk space to external fragmentation is to copy an entire file system onto another disk.
- The original disk is then freed completely, creating one large contiguous free space.

Allocation Methods

- We then copy the files back onto the original disk by allocating contiguous space from this one large hole. This scheme effectively **compacts all free space** into one contiguous space, solving the fragmentation problem. But the cost of compaction is very high.
- **Determining how much space is needed for a file.**
- When the file is created, the total amount of space it will need must be found and allocated.
- If we allocate too little space to a file, we may find that the file cannot be extended.

Allocation Methods

- Two possibilities then exist.
- First, the user program can be terminated, with an appropriate error message. The user must then allocate more space and run the program again. These repeated runs may be costly.
- The other possibility is to find a larger hole, copy the contents of the file to the new space, and release the previous space. This series of actions can be repeated as long as space exists, although it can be time consuming.

Allocation Methods

- Even if the total amount of space needed for a file is known in advance, preallocation may be inefficient.
- A file that will grow slowly over a long period (months or years) must be allocated enough space for its final size, even though much of that space will be unused for a long time.
- The file therefore has a large amount of internal fragmentation.
- some operating systems use a modified contiguous-allocation scheme. Here, a contiguous chunk of space is allocated initially. Then, if that amount proves not to be large enough, another chunk of contiguous space, known as an **extent**, is **added**.

Allocation Methods

Linked Allocation

- With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk.
- The directory contains a pointer to the first and last blocks of the file.
- For example, a file of five blocks might start at block 9 and continue at block 16, then block 1, then block 10, and finally block 25. Each block contains a pointer to the next block.

Allocation Methods

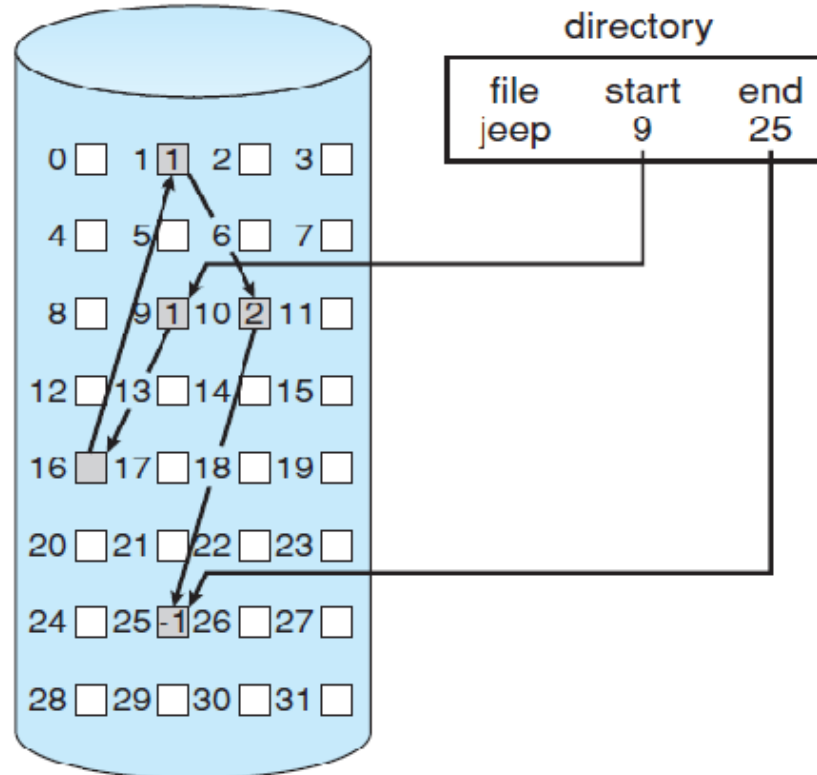


Figure 12.6 Linked allocation of disk space.

Allocation Methods

- To **create** a new file, we simply create a new entry in the directory.
- With linked allocation, each directory entry has a pointer to the first disk block of the file.
- This pointer is initialized to signify an empty file. The size field is also set to 0.
- A **write** to the file causes the free-space management system to find a free block, and this new block is written to and is linked to the end of the file.
- To read a file, we simply **read** blocks by following the pointers from block to block.

Allocation Methods

- **Advantages**

- There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request.
- The size of a file need not be declared when the file is created. A file can continue to grow as long as free blocks are available.
- It is never necessary to compact disk space.

Allocation Methods

- **Disadvantages**
- It can be used effectively only for sequential-access files. To find the i^{th} block of a file, we must start at the beginning of that file and follow the pointers until we get to the i^{th} block. It is inefficient to support a direct-access capability for linked-allocation files.
- Another disadvantage is the space required for the pointers.
 - The usual solution to this problem is to collect blocks into multiples, called clusters, and to allocate clusters rather than blocks. The cost of this approach is an increase in internal fragmentation, because more space is wasted when a cluster is partially full than when a block is partially full.

Allocation Methods

- Another problem of linked allocation is reliability-consider what would happen if a pointer were lost or damaged.
- This error could in turn result in linking into the free-space list or into another file.
- One partial solution is to use doubly linked lists, and another is to store the file name and relative block number in each block.

Allocation Methods

- An important variation on linked allocation is the use of a **file-allocation table (FAT)**.
- A section of disk at the beginning of each volume is set aside to contain the table. The table has one entry for each disk block and is indexed by block number.
- The directory entry contains the block number of the first block of the file. The table entry indexed by that block number contains the block number of the next block in the file.
- This chain continues until it reaches the last block, which has a special end-of-file value as the table entry.

Allocation Methods

- An unused block is indicated by a table value of 0. Allocating a new block to
- a file is a simple matter of finding the first 0-valued table entry and replacing
- the previous end-of-file value with the address of the new block. The 0 is then
- replaced with the end-of-file value.

Allocation Methods

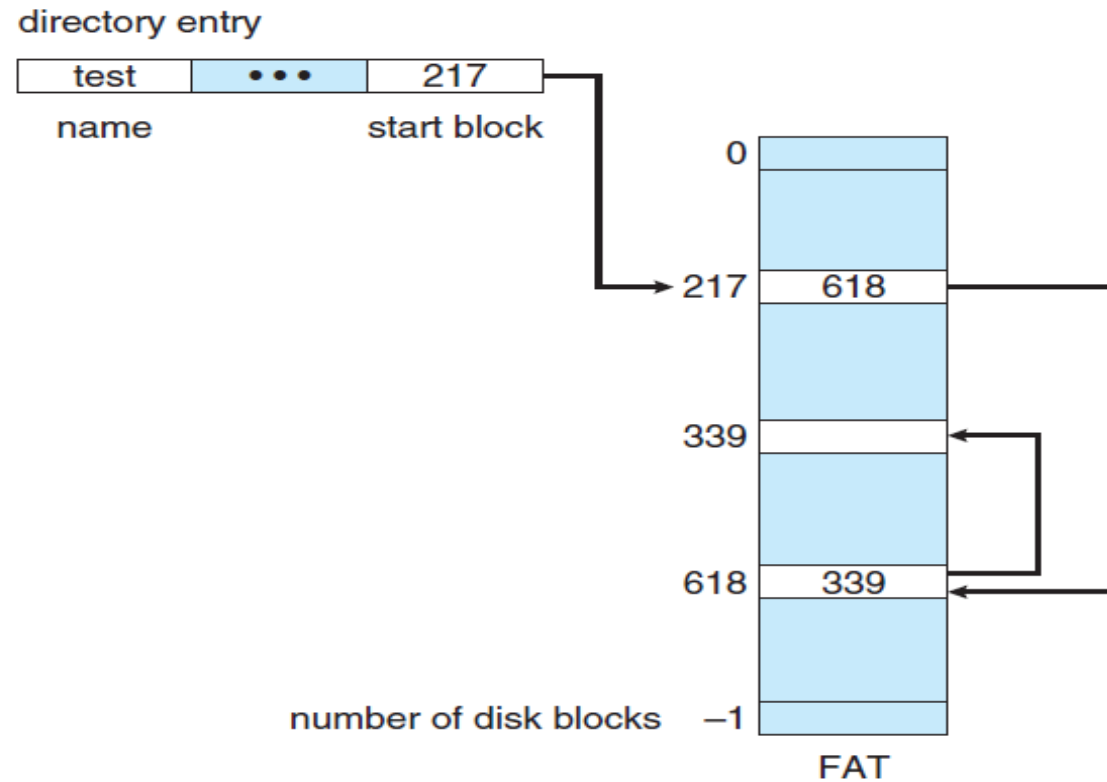


Figure 12.7 File-allocation table.

Allocation Methods

Indexed Allocation

- In the absence of a FAT, linked allocation cannot support efficient direct access, since the pointers to the blocks are scattered with the blocks themselves all over the disk and must be retrieved in order.
- Indexed allocation solves this problem by bringing all the pointers together into one location: the index block.
- Each file has its own index block, which is an array of disk-block addresses. The i^{th} entry in the index block points to the i^{th} block of the file.
- The directory contains the address of the index block. To find and read the i^{th} block, we use the pointer in the i^{th} index-block entry.

Allocation Methods

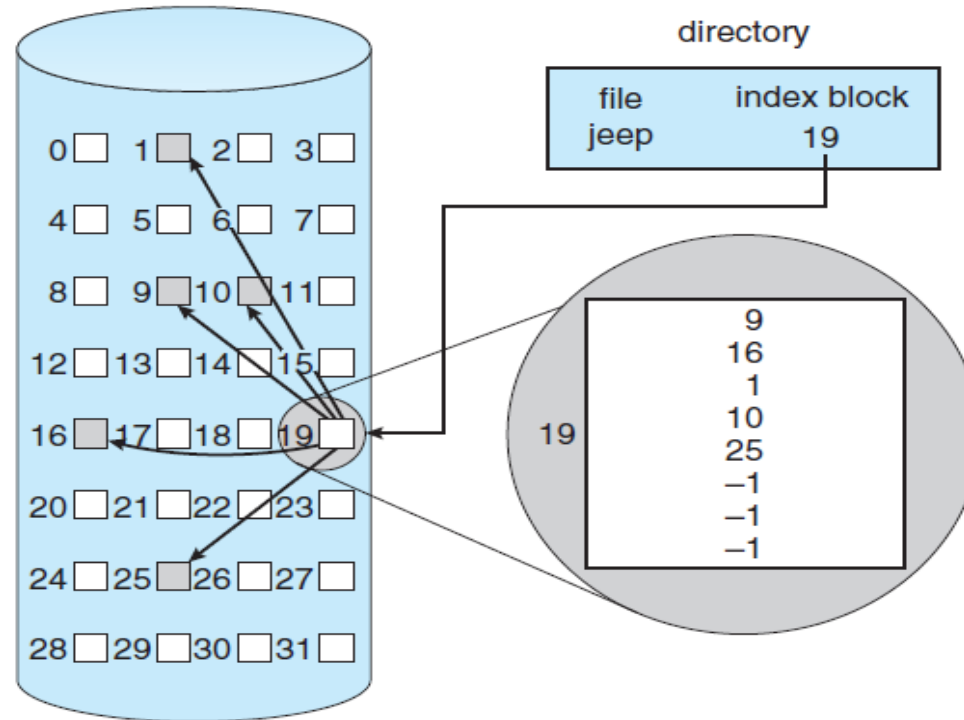


Figure 12.8 Indexed allocation of disk space.

Allocation Methods

- When the file is created, all pointers in the index block are set to null. When the i^{th} block is first written, a block is obtained from the free-space manager, and its address is put in the i^{th} index-block entry.
- **Advantage**
- Indexed allocation supports direct access, without suffering from external fragmentation.
- **Disadvantage**
- The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

Allocation Methods

- **The size of index block-** Every file must have an index block, so we want the index block to be as small as possible.
- If the index block is too small, it will not be able to hold enough pointers for a large file.
Mechanism to deal with this issue is:-
 - **1. Linked scheme-** To allow for large files, we can link together several index blocks.
 - **2. Multilevel index-** A variant of linked representation uses a first-level index block to point to a set of second-level index blocks, which in turn point to the file blocks. To access a block, the operating system uses the first-level index to find a second-level index block and then uses that block to find the desired data block.

Allocation Methods

- **3. Combined scheme-** Another alternative is to keep the first, say, 15 pointers of the index block in the file's inode. The first 12 of these pointers point to direct blocks; that is, they contain addresses of blocks that contain data of the file.
- Thus, the data for small files do not need a separate index block.
- If the block size is 4 KB, then up to 48 KB of data can be accessed directly. The next three pointers point to indirect blocks.
- The first points to a single indirect block, which is an index block containing not data but the addresses of blocks that do contain data.

Allocation Methods

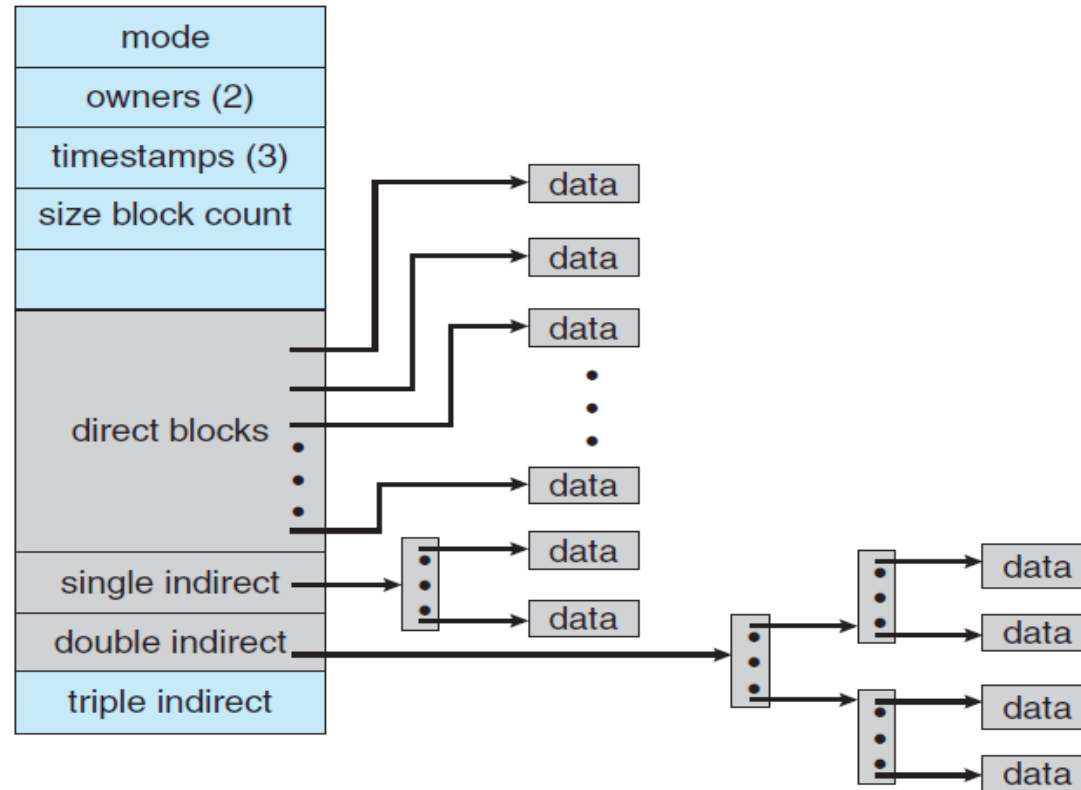


Figure 12.9 The UNIX inode.

Storage Management: Magnetic disks

- Magnetic disks provide the bulk of secondary storage for modern computer systems.
- Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 3.5 inches.
- The two surfaces of a platter are covered with a magnetic material.
- We store information by recording it magnetically on the platters.

Magnetic disks

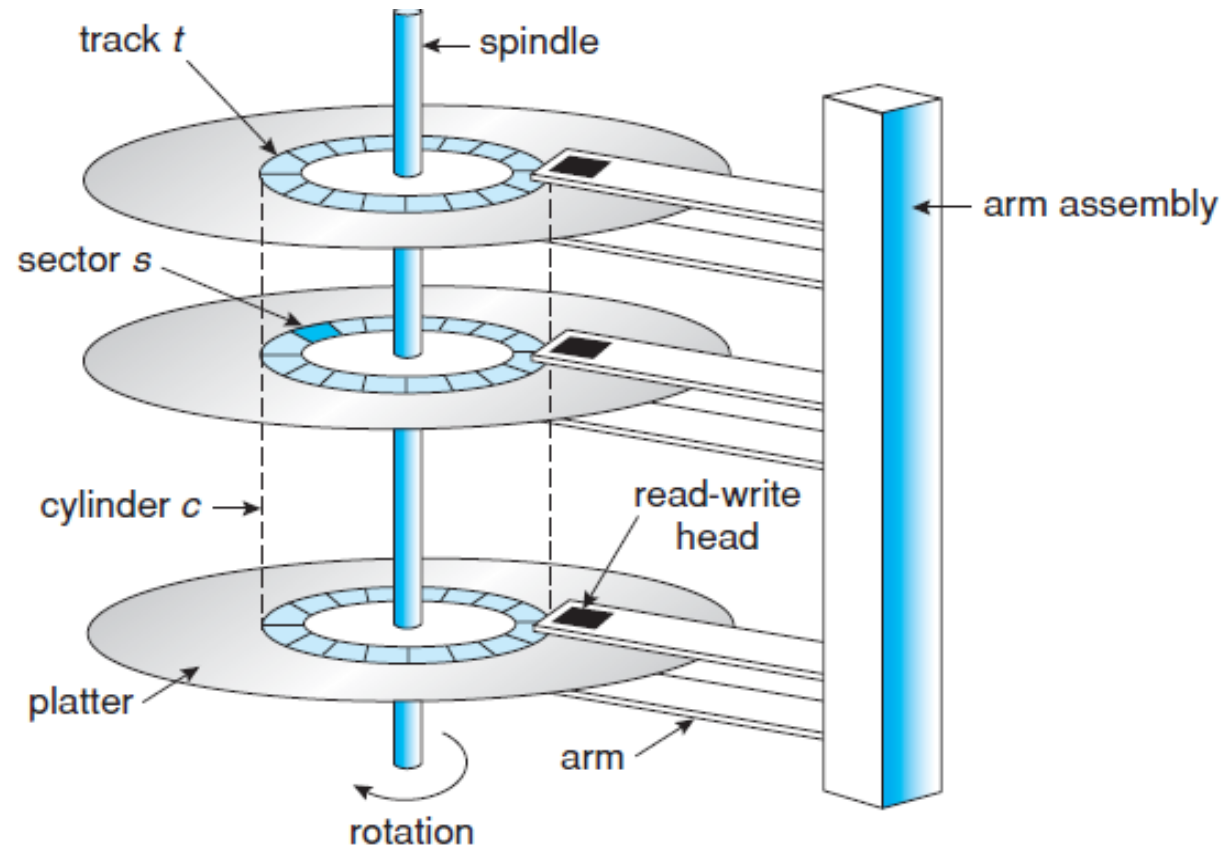


Figure 10.1 Moving-head disk mechanism.

Magnetic disks

- A read–write head “flies” just above each surface of every platter. The heads are attached to a disk arm that moves all the heads as a unit.
- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- The set of tracks that are at one arm position makes up a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors.
- The storage capacity of common disk drives is measured in gigabytes.

Magnetic disks

- When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 250 times per second, specified in terms of rotations per minute (RPM).
- Disk speed has two parts. The **transfer rate** is the rate at which data flow between the drive and the computer. The **positioning time**, or random-access time, consists of two parts: the time necessary to move the disk arm to the desired cylinder, called the **seek time**, and the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.
- Typical disks can transfer several megabytes of data per second, and they have seek times and rotational latencies of several milliseconds.

Magnetic disks

- The disk platters are coated with a thin protective layer, the head will sometimes damage the magnetic surface. This accident is called a head crash.
- A disk can be removable, allowing different disks to be mounted as needed.
- Other forms of removable disks include CDs, DVDs, and Blu-ray discs as well as removable flash-memory devices known as flash drives .
- A disk drive is attached to a computer by a set of wires called an I/O bus. Several kinds of buses are available, including advanced technology attachment (ATA), serial ATA (SATA), eSATA, universal serial bus (USB), and fibre channel (FC).

Magnetic disks

- The data transfers on a bus are carried out by special electronic processors called controllers. The host controller is the controller at the computer end of the bus. A disk controller is built into each disk drive.
- To perform a disk I/O operation, the computer places a command into the host Controller. The host controller then sends the command via messages to the disk controller, and the disk controller operates the disk-drive hardware to carry out the command.

Solid-State Disks

- An SSD is nonvolatile memory that is used like a hard drive.
- SSDs have the same characteristics as traditional hard disks but can be more reliable because they have no moving parts and faster because they have no seek time or latency. In addition, they consume less power.
- They are more expensive per megabyte than traditional hard disks, have less capacity than the larger hard disks, and may have shorter life spans than hard disks, so their uses are somewhat limited.

Solid-State Disks

- There are many variations of this technology, from DRAM with a battery to allow it to maintain its state in a power failure through flash-memory technologies like single-level cell (SLC) and multilevel cell (MLC) chips.

Disk Scheduling

- For magnetic disks, the access time has two major components.
- The **seek time** is the time for the disk arm to move the heads to the cylinder containing the desired sector.
- The **rotational latency** is the additional time for the disk to rotate the desired sector to the disk head.
- The disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

Disk Scheduling

- Whenever a process needs I/O to or from the disk, it issues a system call to the operating system.
- If the desired disk drive and controller are available, the request can be serviced immediately. If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.
- When one request is completed, the operating system chooses which pending request to service next.
- How does the operating system make this choice? Any one of several disk-scheduling algorithms can be used.

Disk Scheduling

FCFS Scheduling

- Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 37, 122, 14, 124, 65, 67, in that order.
- If the disk head is initially at cylinder 53, it will first move from 53 to 98, then to 183, 37, 122, 14, 124, 65, and finally to 67, for a total head movement of 640 cylinders.
- Advantage
 - Simple
- Disadvantage
 - It does not provide the fastest service.

Disk Scheduling

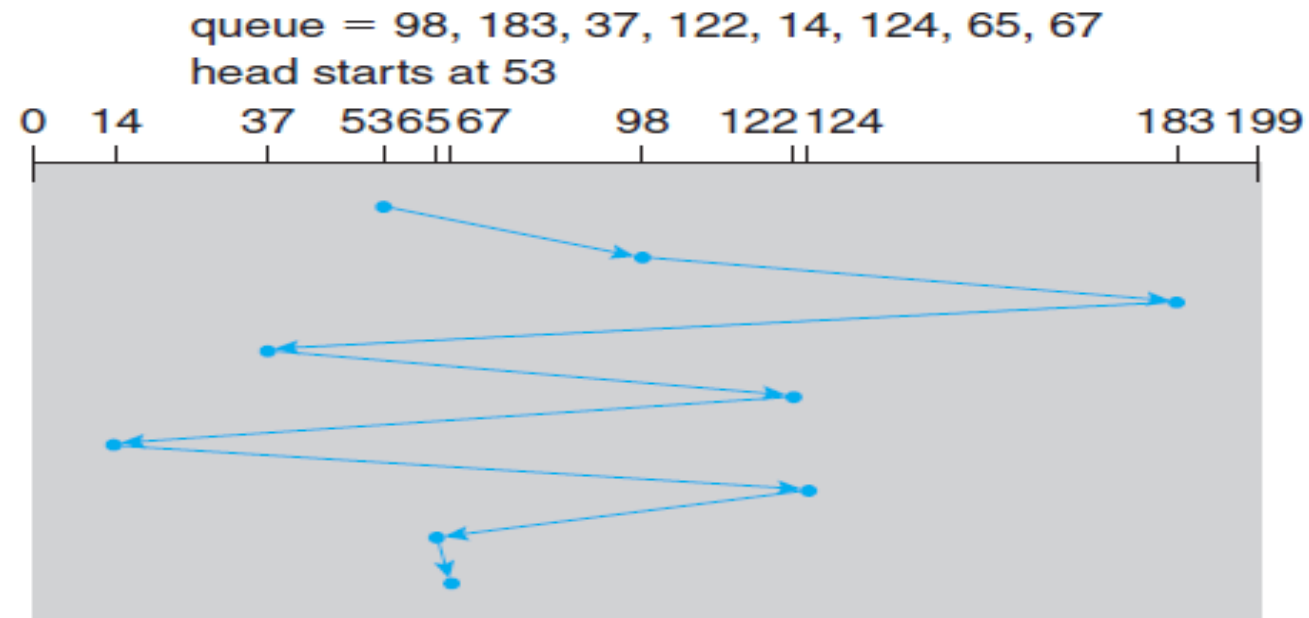


Figure 10.4 FCFS disk scheduling.

Disk Scheduling

SSTF Scheduling(shortest-seek-time-first algorithm)

- The SSTF algorithm selects the request with the least seek time from the current head position.
- SSTF chooses the pending request closest to the current head position.
- Eg:The closest request to the initial head position (53) is at cylinder 65. Once we are at cylinder 65, the next closest request is at cylinder 67. From there, the request at cylinder 37 is closer than the one at 98, so 37 is served next. Continuing, we service the request at cylinder 14, then 98, 122, 124, and finally 183 .
- This scheduling method results in a total head movement of only 236 cylinders.

Disk Scheduling

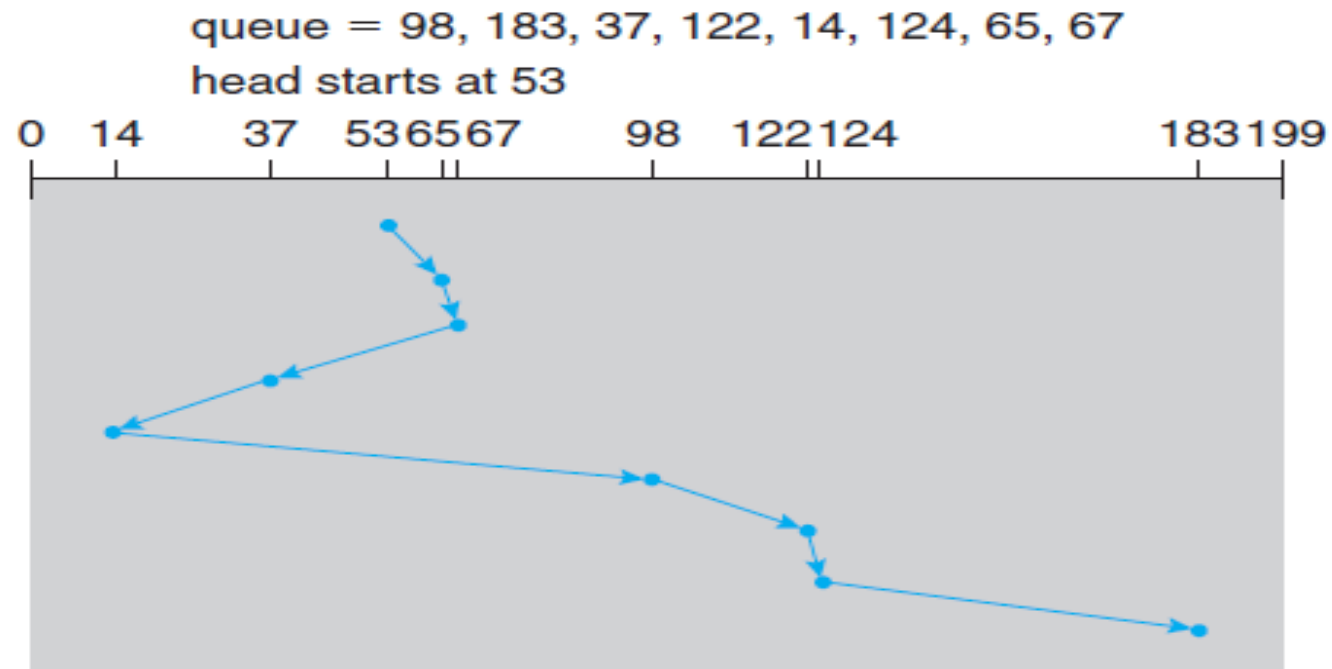


Figure 10.5 SSTF disk scheduling.

Disk Scheduling

- **Advantage**
- Less head movement
- **Disadvantage**
- starvation of some requests-Suppose that we have two requests in the queue, for cylinders 14 and 186, and while the request from 14 is being serviced, a new request near 14 arrives. This new request will be serviced next, making the request at 186 wait. While this request is being serviced, another request close to 14 could arrive. In theory, a continual stream of requests near one another could cause the request for cylinder 186 to wait indefinitely.

Disk Scheduling

SCAN Scheduling

- The disk arm starts at one end of the disk and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.
- At the other end, the direction of head movement is reversed, and servicing continues.
- The head continuously scans back and forth across the disk.
- The SCAN algorithm is sometimes called the **elevator algorithm**.

Disk Scheduling

- Eg: Assuming that the disk arm is moving toward 0 and that the initial head position is again 53, the head will next service 37 and then 14. At cylinder 0, the arm will reverse and will move toward the other end of the disk, servicing the requests at 65, 67, 98, 122, 124, and 183.
- If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.

Disk Scheduling

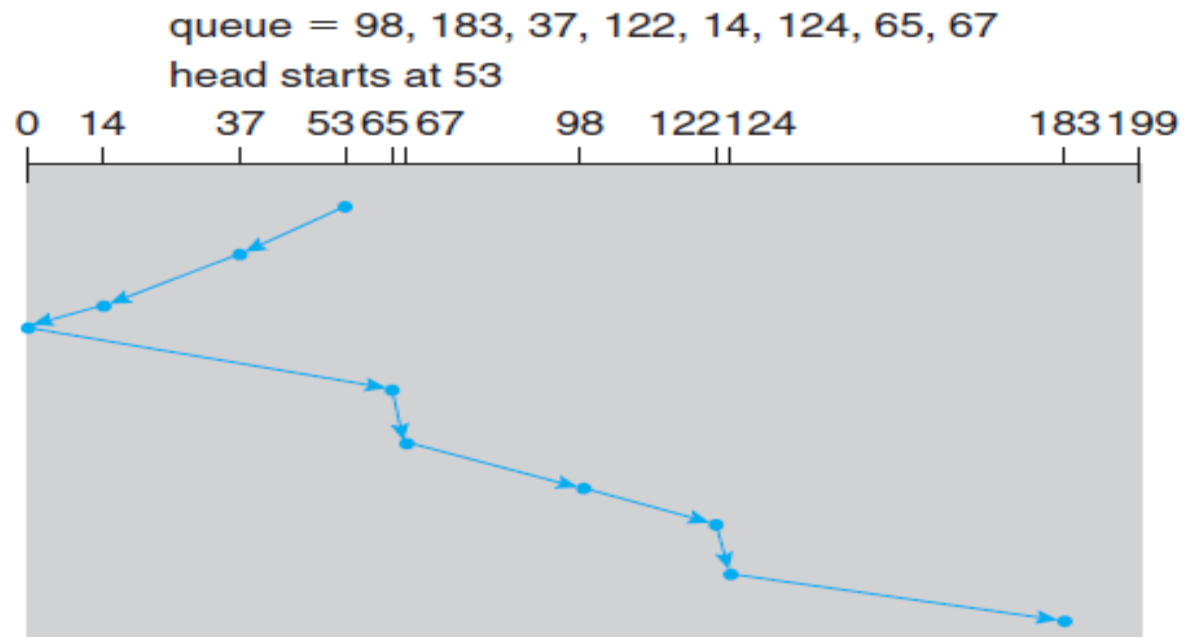


Figure 10.6 SCAN disk scheduling.

Disk Scheduling

C-SCAN Scheduling(Circular SCAN scheduling)

- C-SCAN moves the head from one end of the disk to the other, servicing requests along the way.
- When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.
- The C-SCAN scheduling algorithm essentially treats the cylinders as a circular list that wraps around from the final cylinder to the first one.

Disk Scheduling

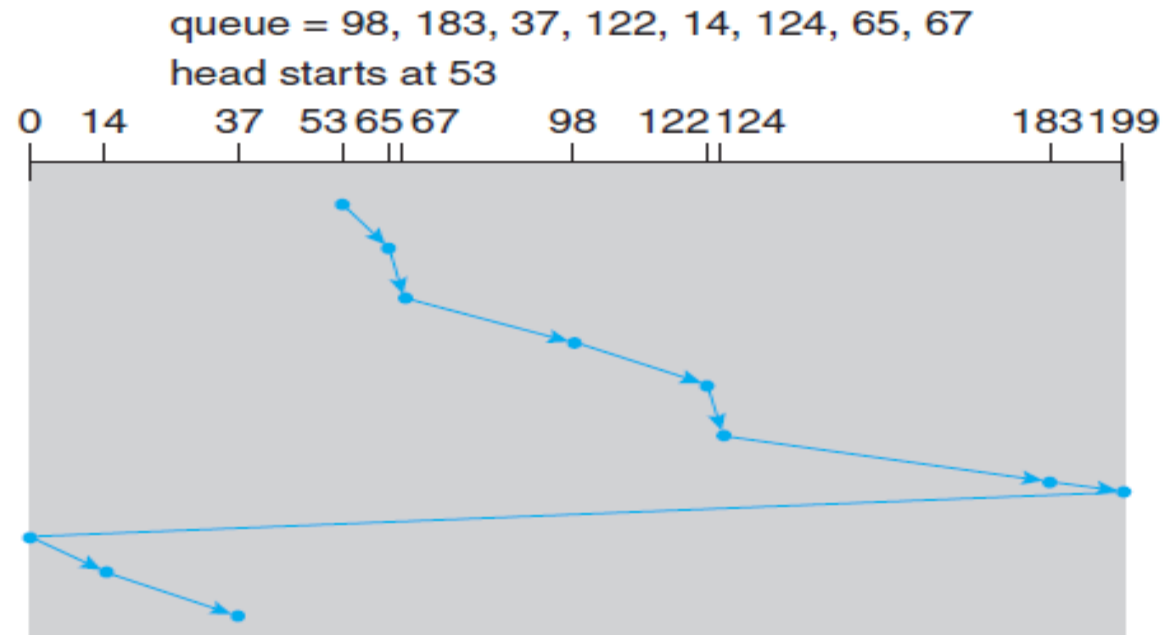


Figure 10.7 C-SCAN disk scheduling.

Disk Scheduling

LOOK Scheduling

- As we described them, both SCAN and C-SCAN move the disk arm across the full width of the disk.
- In LOOK, the arm goes only as far as the final request in each direction. Then, it reverses direction immediately, without going all the way to the end of the disk.
- Versions of SCAN and C-SCAN that follow this pattern are called LOOK and C-LOOK scheduling.

Disk Scheduling

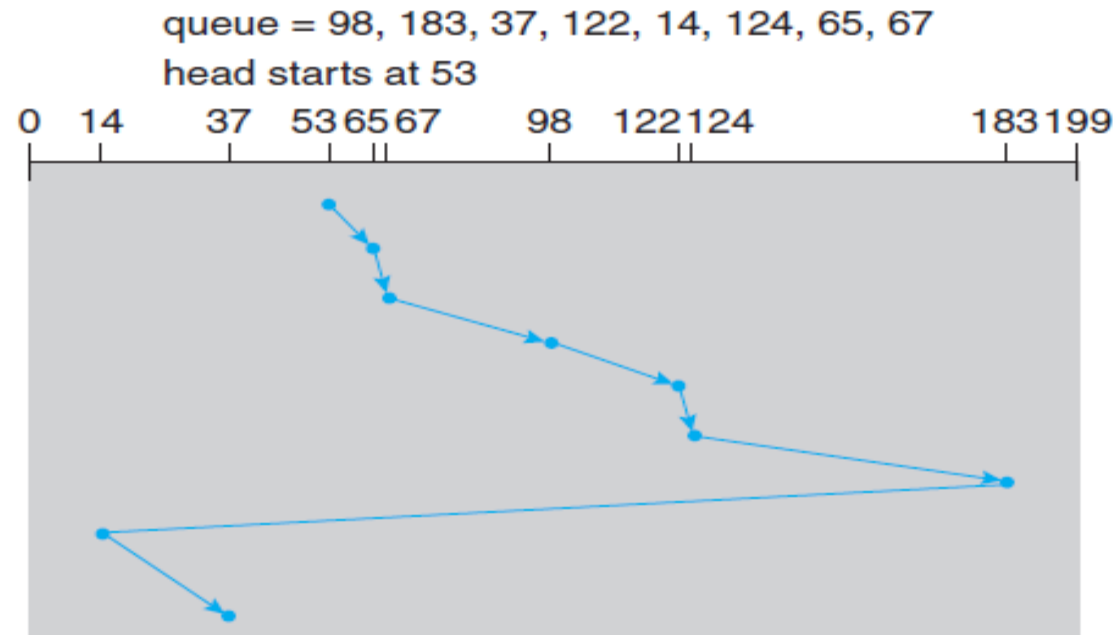


Figure 10.8 C-LOOK disk scheduling.

Disk Formatting

- A new magnetic disk is a blank slate: it is just a platter of a magnetic recording material.
- Before a disk can store data, it must be divided into sectors that the disk controller can read and write. This process is called low-level formatting, or physical formatting.
- Low-level formatting fills the disk with a special data structure for each sector.
- The data structure for a sector typically consists of a header, a data area (usually 512 bytes in size), and a trailer.

Disk Formatting

- The header and trailer contain information used by the disk controller, such as a sector number and an error-correcting code (ECC).
- When the controller writes a sector of data during normal I/O, the ECC is updated with a value calculated from all the bytes in the data area.
- When the sector is read, the ECC is recalculated and compared with the stored value. If the stored and calculated numbers are different, this mismatch indicates that the data area of the sector has become corrupted and that the disk sector may be bad.
- The controller automatically does the ECC processing whenever a sector is read or written.

Disk Formatting

- Most hard disks are low-level-formatted at the factory as a part of the manufacturing process.
- This formatting enables the manufacturer to test the disk and to initialize the mapping from logical block numbers to defect-free sectors on the disk.
- For many hard disks, when the disk controller is instructed to low-level-format the disk, it can also be told how many bytes of data space to leave between the header and trailer of all sectors.

Disk Formatting

- Before it can use a disk to hold files, the operating system still needs to record its own data structures on the disk.
- It does so in two steps. The first step is to partition the disk into one or more groups of cylinders.
- The operating system can treat each partition as though it were a separate disk.
- The second step is logical formatting, or creation of a file system. In this step, the operating system stores the initial file-system data structures onto the disk.

Disk Formatting

- These data structures may include maps of free and allocated space and an initial empty directory.
- To increase efficiency, most file systems group blocks together into larger chunks, frequently called clusters.
- Some operating systems give special programs the ability to use a disk partition as a large sequential array of logical blocks, without any file-system data structures. This array is sometimes called the raw disk, and I/O to this array is termed *raw I/O*.